

Tomato Leaf Disease Classification

From a Small CNN to Fine-Tuned Pretrained Backbones

Deep Learning and Its Applications (CO3133)

Semester 261 – Class CC01 – Group G-02

Assignment 2: Deep Learning on Large-Scale Data and Specialized Tasks

Lecturer: Lê Thành Sách

Students:

Võ Thị Như Quỳnh – 2353041

Mai Trung Kiên – 2352641

Hứa Tuệ Minh – 2450018

Problem Statement & Research Objectives

Problem

Classify a photo of a tomato leaf into one of 8 classes: 7 diseases or healthy.

$$\hat{y} = \arg\max_{k} f_\theta(x)_k, \qquad x \in \mathbb{R}^{3\times 224\times 224}$$

Research objectives

- Compare a small CNN trained from scratch with an ImageNet-pretrained ResNet-18 (frozen and fully fine-tuned).
- Run controlled experiments: augmentation strength, frozen vs full at the same schedule, backbone A vs B.
- Evaluate accuracy, macro-F1, per-class F1, accuracy on field photos, parameters and compute cost.

Challenge

Four diseases make small, similar spots; classes are imbalanced ($9.7:1$); most photos come from a lab, a few from the field.

Dataset & Data Splitting

Dataset: Tomato Leaves (PlantVillage + PlantDoc)

- Hugging Face, revision `c029791`, CC BY 4.0.
- 17,996 RGB images, 8 classes, one label each.
- Dataset proposal (M1): **Approved**.

Split	Images	Purpose
Training	14,404	model training
Validation	1,796	checkpoint selection
Test	1,796	final evaluation, once

Published split, used as is. Split unit: one image.

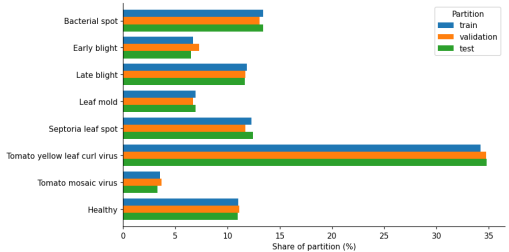


Figure 1: Share of each class in the three partitions. Largest class 6,179 images, smallest 634.

Exploratory Data Analysis: Representative Images & Image Sources

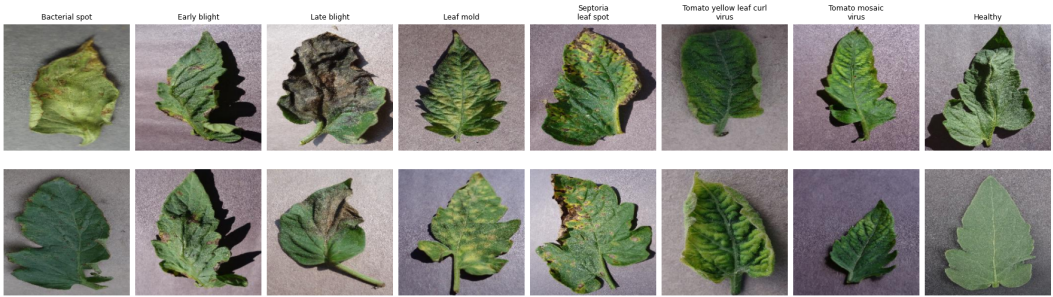


Figure 2: One random training image per class at the 224×224 *model input size.*

- 93.9% of images are 256×256 lab crops: one leaf, grey background.
- Other sizes look like field / web photos. We call them **web-style**.
- Web-style share: 6.6% of train, 4.6% of test (82 images).
- Per class it goes from 3.1% (yellow leaf curl) to 15.0% (mosaic).

Exploratory Data Analysis: Duplicates & Leakage

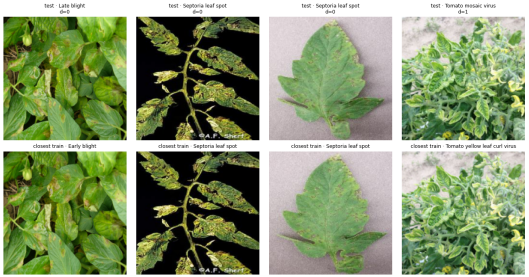


Figure 3: The four closest test images (top) and their closest training images (bottom).

64-bit difference hash, Hamming distance ≤ 4.

88 test images

have a near copy in the training set (4.9%).

25 of them

carry a different label than their training copy.

Decision: keep the published split, and score every model also on a **clean test** of 1,708 images.

Preprocessing Decisions & EDA Summary



Figure 4: Original image, resized to 224×224*, horizontally and vertically flipped.*

Preprocessing

- Resize once to 224×224, cache as uint8.
- ImageNet mean and std (pretrained backbones expect them).
- Training only: random flips; “strong” adds crop + colour jitter.

EDA-based decisions

- Imbalance $\rightarrow$ report macro-F1, select on validation macro-F1.
- Field photos $\rightarrow$ report the 82 web-style images separately.
- Similar spots $\rightarrow$ check spot / blight confusions.

Methodology: Shared Training & Evaluation Pipeline

Implementation

- Own download script, image cache, `TomatoDataset`, CNN, training loop.
- Every setting in one file: `configs/a2.yaml`.
- Pretrained weights from torchvision.
- Same evaluation and timing for every model.

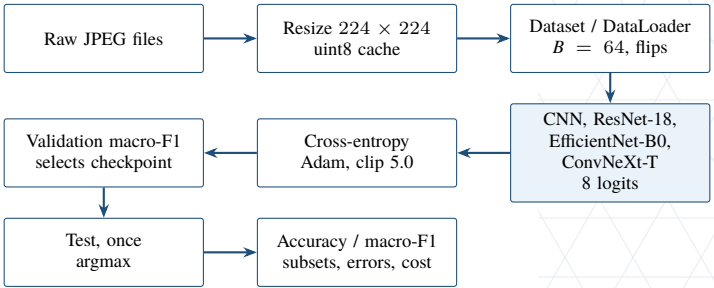


Figure 5: Shared pipeline. Only the shaded model block changes between runs.

Methodology: Models & Fine-Tuning Strategy

Model	Start from	Trained layers	Params	Trainable
CNN (baseline)	random	all	61,384	61,384
ResNet-18 frozen	ImageNet	new 8-class head	11.18M	4,104
ResNet-18 fine-tune	ImageNet	all	11.18M	11.18M
EfficientNet-B0	ImageNet	all (follow-up)	4.02M	4.02M
ConvNeXt-Tiny	ImageNet	all (follow-up)	27.83M	27.83M

Frozen

Head lr 10^{-3}, 6 epochs. Backbone kept in eval mode.
→ **fixed ImageNet features**

Full fine-tune

All layers, lr 10^{-4} ($10\times$ smaller), 4 epochs.
→ **features adapt slowly**

CNN

From scratch, lr 10^{-3}, 25 epochs (M2: 12 was too short).
→ **learns every feature**

Methodology: CNN Baseline Trained from Scratch

Architectural properties

- Four blocks: 3×3 conv, BatchNorm, ReLU, max-pool.
- First conv has stride 2 (fast on CPU).
- Global average pooling, dropout 0.3.
- Total trainable parameters: 61,384.

```
forward()
x = block(3, 16, s=2)(x)    # [B,16,56,56]
x = block(16, 32)(x)        # [B,32,28,28]
x = block(32, 64)(x)        # [B,64,14,14]
x = block(64, 64)(x)        # [B,64,7,7]
x = avgpool(x).flatten(1) # [B,64]
z = fc(dropout(x))        # [B,8]
```

→ **Small and cheap: 3.0 ms per image on the CPU, but it must learn leaf features from 14k images.**

Methodology: ResNet-18, EfficientNet-B0 and ConvNeXt-T

ResNet-18 (main pretrained model)

```
forward()
x = conv1 7x7 s2 + maxpool # [B,64,56,56]
x = layer1..layer4(x)      # [B,512,7,7]
x = avgpool(x).flatten(1)  # [B,512]
z = fc(x)                  # [B,8]
```

Residual blocks $y = F(x) + x$. New head: $512 \times 8 + 8 = 4{,}104$ parameters.

Follow-up backbones (same recipe)

- **EfficientNet-B0**: depthwise conv + squeeze-and-excitation; 4.0M parameters; ImageNet top-1 77.7%.
- **ConvNeXt-T**: 7×7 depthwise conv, LayerNorm; 27.8M parameters; ImageNet top-1 82.5%.
- ResNet-18: ImageNet top-1 69.8%.

$\rightarrow$ **Only the last linear layer is replaced; everything else starts from ImageNet weights.**

Methodology: Experimental Setup

Shared training settings

Setting	Value
Optimizer	Adam, no weight decay
Batch size	64
Gradient clipping	norm 5.0
Seeds	42–46, same split
Checkpoint	best validation macro-F1
Scheduler / early stop	none
Hardware	CPU 14 threads (main)
	1 A40 per run (follow-up)

E1: main comparison (CPU, 20 runs)

CNN vs ResNet-18 frozen vs ResNet-18 fine-tune.

E2 / E4: augmentation ablation

Flip vs strong (crop + colour jitter), with 4 and with 8 epochs.

E3: frozen vs full, same lr and epochs (GPU)

Removes the recipe difference of E1.

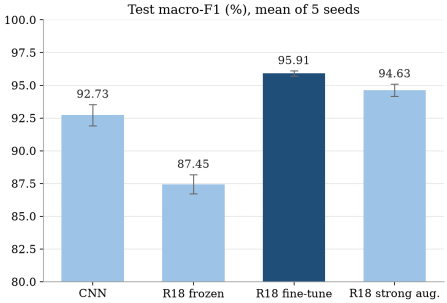
E5: backbone A vs B (GPU)

ResNet-18 vs EfficientNet-B0 vs ConvNeXt-T.

Primary metric: test macro-F1 of the validation-selected checkpoint. **Secondary:** accuracy, web-style accuracy, parameters, time.
A difference is supported when $|\bar{F}_A - \bar{F}_B| > \max(s_A, s_B)$, with s the SD over five seeds (all rules fixed before the runs).

Results: Main Comparison over Five Seeds

Model	Test acc. (%)	Test F1 (%)	Clean F1
CNN	94.69 ± 0.70	92.73 ± 0.81	92.73
R18 frozen	91.12 ± 0.56	87.45 ± 0.74	87.45
R18 fine-tune	**97.13** ± 0.20	**95.91** ± 0.20	96.02
R18 strong aug.	96.28 ± 0.33	94.63 ± 0.47	94.67



Key results

- Fine-tune is best: 95.91 ± 0.20% test macro-F1.
- +3.2 pt over the CNN, in 5/5 seeds.
- The frozen backbone is 5.3 pt *below* the 61K-parameter CNN.
- Rank is the same in all 5 seeds.
- Clean test (no near-duplicates) changes F1 by at most 0.11 pt.

→ leakage does not change any conclusion

Results: Training Behaviour

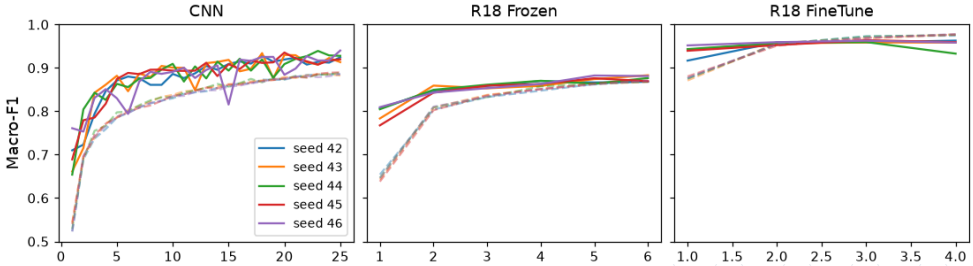


Figure 6: Validation (solid) and training (dashed) macro-F1 for the three main recipes, five seeds (GPU copies; same results as the CPU runs within one SD).

CNN

Selects epoch 20–24 of 25.
→ **budget is now enough**

Frozen

Flat after epoch 4; train ≈ test.
→ **underfits: fixed features**

Fine-tune

Selects epoch 3–4; train F1 98.5 vs test 95.9.
→ **mild overfitting**

Results: Per-Class F1 & Confusions

Class	CNN	Frozen	Fine-tune	Test imgs
Bacterial spot	0.955	0.901	**0.964**	241
Early blight	0.835	0.739	**0.943**	117
Late blight	0.901	0.869	**0.962**	209
Leaf mold	0.897	0.832	**0.937**	125
Septoria leaf spot	0.936	0.880	**0.960**	223
Yellow leaf curl	0.986	0.978	**0.993**	625
Mosaic virus	**0.932**	0.825	0.931	59
Healthy	0.975	0.971	**0.982**	197

Early blight

Hardest class; fine-tuning +10.8 pt over the CNN.
→ **the largest per-class gain**

Spot / blight confusion

Early blight, leaf mold → late blight: 8% (CNN), about 1% after fine-tuning.

Mosaic → yellow leaf curl

Stays in every model (3–10%); only 59 test images.

Results: Accuracy vs. Computational Cost

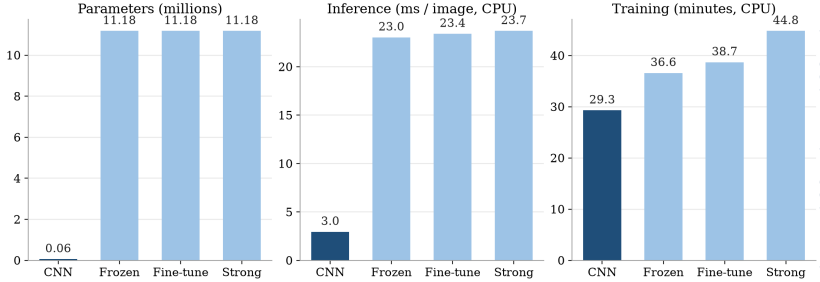


Figure 7: Parameters, CPU inference time (batched, median of 2 passes after 1 warm-up) and total training time; one CPU, 14 threads.

- **Fine-tune costs 8× the CNN at inference** (23.4 vs 3.0 ms) and +32% training time for +3.2 pt macro-F1.
- The frozen model trains 4,104 parameters but still runs the whole backbone: 5× the CNN per epoch, lowest F1.
- On one A40: 1.5 min to fine-tune ResNet-18, about 0.5 ms per image (data loading is the bottleneck).

Ablation: Strong Augmentation (E2 and E4)

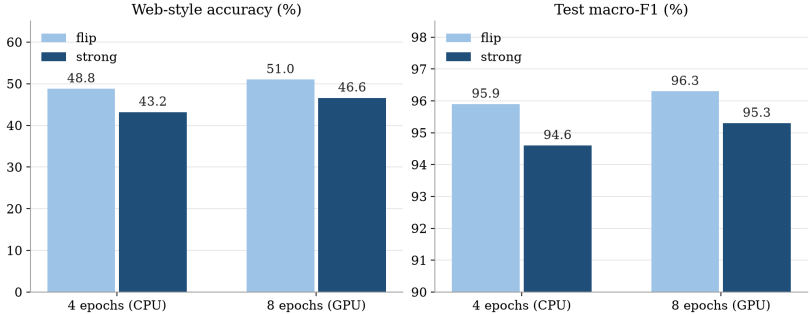


Figure 8: ResNet-18 fine-tune with flip vs strong augmentation, mean of 5 seeds. Strong = flip + random resized crop + colour jitter.

Decision: not supported, twice

Web-style −5.6 pt (4 ep) and −4.4 pt (8 ep); better in only 0/5 and 1/5 seeds.

Not a budget problem

With 8 epochs the strong runs pick epoch 6–7.
→ **generic crops do not look like field photos**

Ablation: Frozen vs Full Fine-Tuning at the Same Schedule (E3)

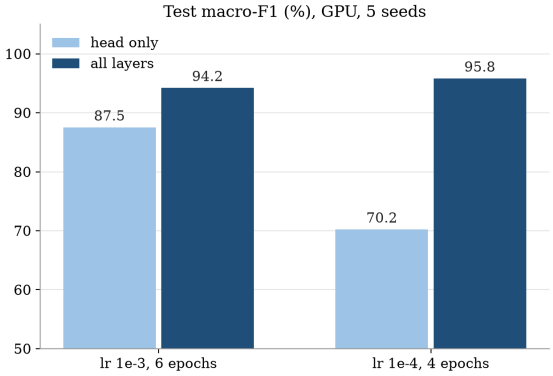


Figure 9: Only the trained layers change inside each pair.

Question

In E1 frozen and full also differ in lr and epochs. Is the gap caused by freezing?

Result: supported

Full wins by 6.8 pt and 25.7 pt, in 5/5 seeds at both schedules.

→ **freezing is the cause**

Interaction

The head alone needs lr 10^{-3}; full fine-tuning prefers 10^{-4}.

→ **each E1 recipe was the better one**

Follow-Up: Backbone A vs B (E5)

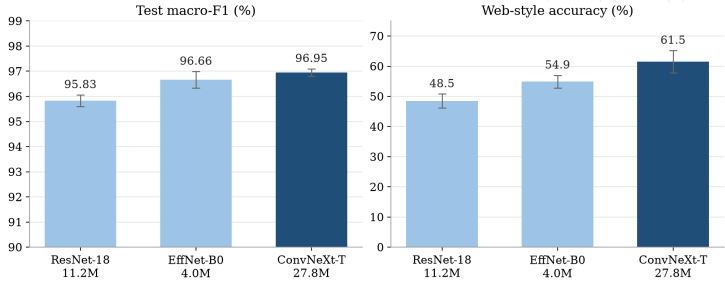


Figure 10: Same recipe (full fine-tune, lr 10^{-4}, 4 epochs, flip), one A40 per run, mean $\pm$ SD of 5 seeds.

Supported

Both beat ResNet-18 in 5/5 seeds (+0.8 and +1.1 pt).

Field photos

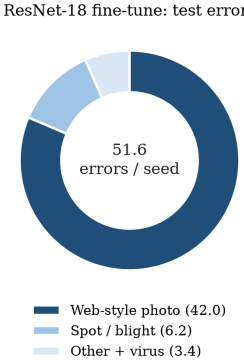
ConvNeXt-T: 48.5 $\rightarrow$ 61.5% web-style accuracy.

$\rightarrow$ **biggest gain where it matters**

Trade-off

EfficientNet-B0: 2.8$\times$ fewer parameters; ConvNeXt-T: 3–3.5$\times$ slower on CPU.

Analysis: Error Analysis



81.4%

of the fine-tuned model's errors are web-style photos (42.0 of 51.6 per seed).

51%

of the 82 web-style test images are wrong, against 0.6% of the 1,714 lab images.

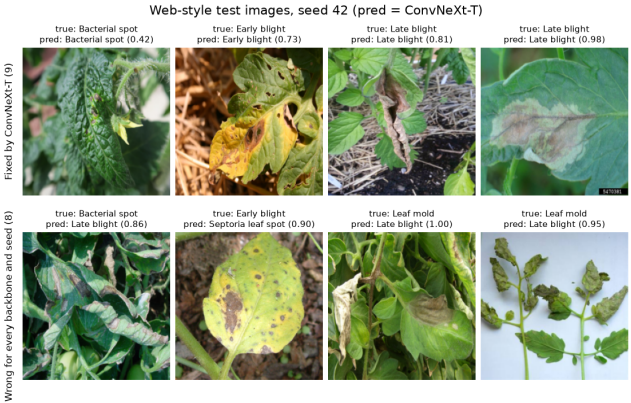
−74% vs −29%

From CNN to fine-tune, lab errors drop 74% but web-style errors only 29%.

→ **fine-tuning fixes lab errors, not the domain gap**

Cause (hypothesis): only 6.6% of training images are web-style. Ideas: more field images, crop the leaf first, field-like augmentation.

Analysis: Qualitative Results



Fixed by ConvNeXt-T

One leaf or one lesion large in the frame.

Wrong for every model

8 of 82 web-style images: many leaves, fruit, cut plant; often confidence ≥ 0.9. $\rightarrow$ **confidence cannot flag field photos**

Figure 11: Web-style test images, seed 42. Top: ResNet-18 wrong, ConvNeXt-T right. Bottom: wrong for all backbones and seeds.

Discussion & Conclusion

Main comparison

Fine-tune 95.91% vs CNN 92.73% vs frozen 87.45% macro-F1; same rank in 5/5 seeds.
→ **pretraining helps only with fine-tuning**

Frozen vs full (E3)

Gap stays at the same lr and epochs (+6.8 pt).
→ **freezing is the cause**

Augmentation (E2, E4)

Strong augmentation hurts even with 8 epochs.
→ **generic transforms are not field photos**

Backbone (E5)

ConvNeXt-T 96.95% F1 and 61.5% on web-style photos.
→ **better features carry over to the field**

Cost

CNN 8× faster than ResNet-18 on CPU; on a GPU all are $\leq$ 1.1 ms per image.
→ **accuracy has a compute price**

Limits

Web-style subset small (82) and defined by size; one recipe per model; near-duplicates kept.

Fine-tuning a pretrained backbone wins on lab photos; the real open problem is the gap between lab and field photos.

Even the best model still misses 38% of the web-style images.

AI Usage Disclosure

Claude and ChatGPT (whole project, all members)

Ideas, concept explanations, generic code examples, wording of notebook text.

Claude Code, Claude Opus 5.5 (Võ Thị Như Quỳnh, 8–9 Nov 2026)

Per-run device option, EfficientNet-B0 / ConvNeXt-T wrappers, running the 45 GPU runs, `03_analysis.ipynb`, first drafts of the report and these slides.
→ **every number comes from saved run outputs and executed notebooks**

Code, report, slides and full disclosure (`AI_USAGE.md`): `https://quin210.github.io/dl-co3133/`